Measuring Reinforcement Learning Against an Exact Optimum: Reward Design, Initialisation and Algorithm Choice for a Retail Shelf-Restocking Robot

Sakshi Bansal
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India
Reg. No. 23BPS1012
sakshi.bansal2023@vitstudent.ac.in

Dr. D. Jeya Mala
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India
jeyamala.d@vit.ac.in

***Abstract*—Hand-designed rewards and default settings shape what a reinforcement-learning agent learns, yet realistic problems offer no optimum to compare against. This paper studies a shelf-restocking robot in a store small enough to solve exactly, and uses the exact optimum to measure both the reward and the learners. The state is reduced to 768 Markov states, an exact model is validated against the simulator, and every policy, tabular or neural, is evaluated exactly on ten store layouts. Dynamic programming shows that the project's original reward pays for walking back and forth near a shelf. Its optimal policy travels 6% further on average (up to 17%), at a cost of only 0.3% of the objective, whereas potential-based versions of the same guidance leave the optimal policy unchanged. Among nine learners trained under four rewards with identical budgets, DQN and Double DQN come closest to the optimum (normalised score 0.96), ahead of PPO (0.79) and every tabular method (at most 0.73). Both bonuses in the original reward help DQN, and their potential-based forms recover about half of that gain. For tabular learners the initial Q-value matters more than the reward: since potential-based shaping is equivalent to initialisation, moving the starting value from 0 to -150 triples Q-Learning's score, and direct counts reveal the failure modes on either side. Finally, REINFORCE's argmax policy scores less than half of its stochastic policy, so policy-gradient agents must be evaluated as stochastic policies.**

***Index Terms*—reinforcement learning, reward shaping, dynamic programming, Q-value initialisation, deep Q-network, retail automation, inventory restocking**

I. Introduction

A shelf that runs empty loses sales until somebody refills it. A restocking robot that decides for itself where to go and when to refill can close that gap, but only if it learns a good policy, and a reinforcement-learning agent learns whatever its reward pays for. The reward is usually designed by hand, with extra terms meant to speed learning up: a bonus for restocking, a bonus for heading toward the emptiest shelf. Whether such terms change what the agent ultimately learns, and how close the agent comes to the best possible policy, are questions that most applied studies cannot answer, because their problems are too large to solve exactly.

This project takes the opposite route. The store, a 4×4 grid with a depot, two shelves and a robot that carries two units, is deliberately small enough that the optimal policy can be computed exactly by dynamic programming [4]–[6]. That optimum is then used as a measuring instrument for everything else. Every learned policy, whether a Q-table or a neural network, is evaluated exactly against it, with no sampling noise, on ten store layouts of very different difficulty.

The study builds on the project's earlier reviews, which compared ten algorithm configurations on the same task. Re-examining that work against the exact optimum changed several of its conclusions (Section VII) and widened the question. Besides asking which algorithm scores highest, this report asks what the reward does to the optimum and why each learner stops where it does.

The contributions are as follows.

- *An exact reference for a restocking MDP.* The robot's state is reduced to a 768-state Markov tuple, and an exact model is validated against the simulator. Any policy can then be evaluated exactly and scored on a common 0–1 scale across layouts (Section III).
- *An exact audit of the reward.* The proposal's bonus for approaching the emptiest shelf is not potential-based, and its optimal policy does exploit the resulting loop, walking 6% further on average and up to 17%, at a cost of only 0.3% of the objective. Potential-based versions of the same guidance leave the optimum unchanged in every state. The discount factor, the travel price and a travel-time bound on service are examined in the same way (Section IV).
- *A controlled comparison of nine learners under four rewards.* All learners receive the same budget, state and evaluation, and the time limit is treated consistently. DQN trained on the proposal reward comes within a few percent of the optimum. Both proposal bonuses help it, and their potential-based forms recover about half of that help (Section VI).

- *An explanation of the tabular learners' shortfall.* Combining the exact values with Wiewiora's equivalence between shaping and initialisation [14] shows that the starting Q-value, not the reward, limits the tabular learners. Changing only that value triples Q-Learning's score, and the two failure modes on either side of the best value can be observed directly (Section VI-D).

All code, results and notebooks are available at https://github.com/sbsakshi/rl-retail-restocking.

II. Related Work

A. Reinforcement learning for inventory and warehouse robots

Inventory control is a long-standing test bed for sequential decision making. Gijsbrechts et al. showed that a tuned A3C agent can match strong heuristics and approximate dynamic programming on lost-sales, dual-sourcing and multi-echelon problems [20], and Boute et al. survey the design choices such agents require [21]. Liu et al. coordinate the echelons of a supply chain with heterogeneous-agent PPO [22]. Closer to the store floor, Jullien et al. built RetaiL, a grocery restocking environment from real sales data, and trained a distributional DQN to reduce waste [23]. On the robotics side, Zhu et al. schedule robots in a mobile fulfilment system with deep RL and metaheuristics [24], and Li et al. combine PPO with Dijkstra planning for warehouse navigation [25]. These studies decide *how much* to order or *how* to move; the present task couples the two, since the robot's route determines which shelf can be refilled before it runs out. None of them can say how far their policies are from optimal, because their problems are too large to solve exactly. This study deliberately chooses a store small enough to solve, so that every learned policy can be measured against the true optimum.

B. Reward shaping

Shaping adds reward for progress toward a goal to speed up learning. Randløv and Alstrøm found that rewarding progress without penalising regress let a bicycle agent profit from riding in circles [15], a simple instance of what Amodei et al. call reward hacking [16]. Ng, Harada and Russell proved that shaping of the form $\gamma\Phi(s') - \Phi(s)$ preserves the optimal policy, and that this form is also necessary for the guarantee to hold in every MDP [13]. Wiewiora then showed that, for tabular TD learners, adding such a term is equivalent to initialising $Q(s, a)$ to $\Phi(s)$ [14]. That equivalence turns out to explain most of what shaping does, and does not do, for the tabular learners in Section VI.

C. Evaluation practice

Henderson et al. showed that deep RL results often fail to reproduce across random seeds and implementations [18], and Agarwal et al. argued for interval estimates over many runs in place of point estimates [19]. Pardo et al. showed that treating a time limit as termination corrupts value estimates and recommended bootstrapping at the cut [17]. Van Hasselt traced Q-Learning's weakness in stochastic problems to the overestimation introduced by the max operator [8]. All four points apply directly to the agents compared here, and Section V adopts the corresponding practices.

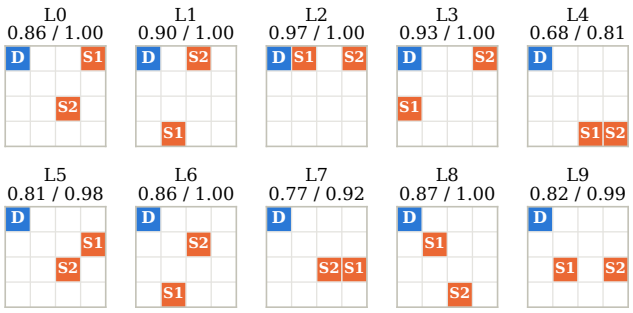


Fig. 1. The ten store layouts (D: depot; S1, S2: shelves). Each title gives the layout, its optimal service level and the travel-time bound of Section IV-E.

III. The Restocking Problem

A. Store and robot

The store is a 4×4 grid with a depot at $(0, 0)$ and two shelves, each holding up to three units. The robot carries at most two units. At every step it chooses one of seven actions: move up, down, left or right, restock the shelf it stands on, reload at the depot, or wait. Restocking moves as many units as fit from the robot onto the shelf; reloading fills the robot, but only at the depot. After the action, each shelf that still has stock independently loses one unit with probability $p = 0.12$, so the two shelves together see on average one sale every four steps. A run is cut after $T = 200$ steps. The store does not close at that point, so the cut is a time limit and not a terminal state; Section V returns to why this distinction matters.

Where the two shelves stand is decided by a layout seed. Ten layouts (seeds 0–9) are used throughout, and each acts as one independent replicate of every experiment. They differ a great deal in difficulty (Fig. 1): when both shelves are close to the depot almost no sale is ever lost, but when both are five or six cells away the robot cannot keep up with demand.

B. State

The Gymnasium observation has eight entries: robot position, units carried, the two stock levels, the number of steps since each shelf was last restocked, and a time-of-day clock. Only the first five affect what happens next. Time since a visit adds nothing once the stock level itself is observed, and demand does not vary with the time of day. The tuple

$$s = (x,\ y,\ c,\ \sigma_1,\ \sigma_2) \tag{1}$$

of position, carried units and stock levels is therefore a Markov state, with $4 \cdot 4 \cdot 3 \cdot 4 \cdot 4 = 768$ values. Every agent in this study, tabular or neural, observes exactly this tuple. In Review II the tabular agents also carried a two-valued time-of-day bit, which doubled their table without telling them anything.

C. What the store pays for

The objective is defined first, before any reward used for training. The store loses sales when a shelf is empty and pays for robot travel, so the objective reward is

$$r_{\text{obj}} = -8\, n_{\text{empty}} - 0.1\, m, \tag{2}$$

where n_{empty} counts shelves at zero stock after demand and $m \in \{0, 1\}$ records whether the robot moved. A sale is lost only when a customer arrives at an empty shelf, which happens with probability p per step, so the expected number of lost sales is proportional to the number of empty shelf-steps. The weight 0.1 says that 80 moves cost as much as one shelf standing empty for one step; Section IV-D examines what that choice implies. Every policy in this paper is scored on (2), whatever reward it was trained on.

D. The proposal reward

The project proposal trained agents on a richer signal:

$$\begin{aligned} r_{\text{prop}} = r_{\text{obj}} - 0.2 - 1 \cdot \mathbb{1}[\text{invalid}] + 10 \cdot \mathbb{1}[\text{restock}, \sigma > 0] \\ + 5 \cdot \mathbb{1}[\text{restock}, \sigma = 0] + 2 \cdot \mathbb{1}[\text{closer}], \end{aligned} \tag{3}$$

with a constant step cost, a penalty for a restock or reload that cannot happen, a bonus for restocking (halved if the shelf was already empty), and a bonus whenever a step reduces the distance to the lowest-stock shelf.

The closer bonus can be gamed. A step toward the target earns $2 - 0.1 - 0.2 = +1.7$, while a step away costs only -0.3, so walking back and forth earns $+1.4$ every two steps where waiting earns -0.4. A reward that pays for progress without charging for regress makes loops profitable; Randløv and Alstrøm observed exactly this when a shaped bicycle controller learned to ride in circles [15]. Whether the loop is worth exploiting once stockouts are taken into account is not obvious, and Section IV-B settles it exactly.

E. Potential-based alternatives

Ng, Harada and Russell showed that a shaping term of the form

$$F(s, s') = \gamma\, \Phi(s') - \Phi(s) \tag{4}$$

leaves the optimal policy unchanged for any bounded potential Φ, because the shaped optimal action values are $Q^*(s, a) - \Phi(s)$ [13]. Two potentials are tested. The first keeps the proposal's guidance signal, the same target and the same magnitude, in potential form:

$$\Phi_d(s) = -2\, \|\text{pos}(s) - \text{shelf}_{\arg\min_i \sigma_i}\|_1. \tag{5}$$

The second adds the restock bonus in potential form,

$$\Phi_{ds}(s) = \Phi_d(s) + 5\,(\sigma_1 + \sigma_2), \tag{6}$$

so a two-unit restock pays about +9.5 at the moment it happens, close to the proposal's +10. The price is that each random sale now also produces a shaping penalty of -5γ. The four training rewards are referred to as *Proposal*, *Objective*, *PBRS-d* and *PBRS-ds*.

F. Exact model

Given (s, a), everything except demand is deterministic, and demand has at most 2^2 outcomes. The model enumerates them into arrays of successor indices, probabilities and reward components, so any of the rewards above is a linear combination of stored arrays. The model was checked against the simulator in three ways, all kept as unit tests: every successor observed in 6,000 random transitions exists in the model with the same reward under all four rewards; empirical successor frequencies over 20,000 draws agree with the model's probabilities to within four standard errors; and the new simulator, run with the proposal reward, reproduces the Review II environment to 10^{-15} for random action sequences.

G. Exact evaluation and the normalised score

Because every agent acts on the Markov state, any trained agent, whether a Q-table or a neural network, can be written as a policy π over the 768 states. Pushing the start-state distribution forward, $d_{t+1} = d_t P_\pi$, gives the exact expected value of every metric over a 200-step run with no sampling noise. The model is used only to measure; no learner ever sees it. As a cross-check, every final policy is also simulated on 100 demand sequences never used in training; Section VI shows that the two measurements agree.

Raw returns from different layouts cannot be averaged, because the layouts differ so much in difficulty. Each policy is therefore placed between a uniform-random policy and the best achievable behaviour:

$$\text{score}(\pi) = \frac{J(\pi) - J(\pi_{\text{rand}})}{J_T^* - J(\pi_{\text{rand}})}, \tag{7}$$

where J is the expected 200-step sum of (2) from reset and J_T^* is its maximum, obtained by backward induction over the 200 steps. A score of 1 is the best any policy can do in expectation.

IV. What the Exact Optimum Shows

A. Value and policy iteration

Value iteration (VI) on the objective with $\gamma = 0.95$ converges to a change below 10^{-10} in 441–487 sweeps, about 0.05 s per layout. Policy iteration (PI), with exact policy evaluation by a linear solve, stops after 13–18 improvement steps. The two value functions agree to 2×10^{-9}. Their greedy actions coincide in only 88–96% of states, however, which matches the unexplained 88.9% reported in Review II. The explanation is ties: between 242 and 331 of the 768 states have two or more actions whose values differ by less than 10^{-6}, usually two moves of equal length toward the same shelf. Counting any optimal action as agreement, PI's policy is optimal for VI in every state of every layout (Table I).

B. Does the training reward change what is optimal?

For each training reward the optimal policy was computed and then scored on the objective (Table II). Three findings follow.

The closer bonus does pay for itself in travel. The proposal reward's optimal policy walks 6.1% further than the objective's

TABLE I

EXACT SOLUTION OF EACH LAYOUT UNDER THE OBJECTIVE. SL^*: OPTIMAL SERVICE LEVEL; BOUND: TRAVEL-TIME BOUND (SEC. IV-E); TRAVEL: MOVES PER RUN UNDER π^*; AGREE: SHARE OF STATES WHERE VI AND PI PICK THE SAME ACTION; TIES: STATES WITH MORE THAN ONE OPTIMAL ACTION.

L	shelves	SL^*	bound	travel	SL_{rand}	VI	PI	agree	ties
0	(3,0), (2,2)	0.856	1.000	150	0.161	471	14	0.917	293
1	(1,3), (2,0)	0.899	1.000	143	0.164	464	14	0.931	287
2	(1,0), (3,0)	0.969	1.000	120	0.166	441	13	0.939	242
3	(0,2), (3,0)	0.930	1.000	130	0.164	456	13	0.880	305
4	(2,3), (3,3)	0.683	0.807	165	0.154	487	18	0.958	298
5	(3,1), (2,2)	0.807	0.983	156	0.161	477	18	0.892	331
6	(1,3), (2,1)	0.856	1.000	150	0.164	471	16	0.926	298
7	(3,2), (2,2)	0.769	0.917	157	0.159	480	17	0.930	316
8	(1,1), (2,3)	0.868	1.000	152	0.166	469	16	0.889	320
9	(1,2), (3,2)	0.816	0.990	154	0.162	476	15	0.919	282

TABLE II

OPTIMAL POLICY OF EACH TRAINING REWARD, SCORED ON THE OBJECTIVE (MEANS OVER 10 LAYOUTS). EXTRA TRAVEL IS RELATIVE TO THE OBJECTIVE'S OPTIMUM, WITH THE RANGE OVER LAYOUTS; THE LAST COLUMN IS THE SHARE OF STATES IN WHICH THE POLICY'S ACTION IS OPTIMAL FOR THE OBJECTIVE.

optimal policy of	score	SL	travel	extra travel (%)	opt. (%)
Proposal	0.9922	0.8440	156.2	+6.1 [+2.1, +16.8]	96.3
Proposal − closer	0.9909	0.8429	148.1	+0.3 [−3.1, +4.1]	96.6
Objective + restock	0.9909	0.8429	148.1	+0.3 [−3.1, +4.1]	96.6
Objective + closer	0.9907	0.8428	156.6	+6.4 [+2.5, +17.7]	97.1
Objective	0.9946	0.8453	147.8	+0.0 [+0.0, +0.0]	100.0
PBRS-d	0.9946	0.8453	147.8	+0.0 [+0.0, +0.0]	100.0
PBRS-ds	0.9946	0.8453	147.8	+0.0 [+0.0, +0.0]	100.0

on average and 16.8% further on layout 2. Removing only the closer bonus brings travel back to within 0.3% of the objective's optimum on average (between −3.1% and +4.1% on individual layouts), and adding only the closer bonus to the objective reproduces the extra travel. The loop described in Section III therefore matters in practice: an optimal agent under the proposal reward takes it whenever the shelves can spare the time.

The damage to the objective is small. The proposal reward's optimal policy still reaches a normalised score of 0.992, against 0.995 for the objective's own optimal policy. The stockout penalty dominates every other term, so an agent cannot afford to loop when a shelf is about to run out. The restock bonus alone barely changes travel, but it makes the optimal action wrong in about 3% of states: it rewards a top-up for its own sake rather than for the sale it protects. The constant step cost and the invalid-action penalty change nothing, as rows 2 and 3 of Table II are identical.

Potential-based shaping is exactly invariant here. Under both PBRS-d and PBRS-ds the greedy action of the shaped optimum is optimal for the objective in all 768 states of all ten layouts, and the shaped action values equal $Q^* - \Phi$ to machine precision, as the theorem predicts [13].

C. The discount factor

The quantity of interest is the undiscounted 200-step sum, so γ is a modelling choice, not part of the problem. Solving the stationary problem for several discounts and scoring each policy against J_T^* gives mean scores of 0.983 ($\gamma = 0.8$), 0.993 (0.9), 0.995 (0.95), 0.996 (0.98) and 0.999 (0.99). A discount of 0.95, an effective horizon of about 20 steps or two to three round trips, gives up half a percent of the achievable improvement and is used for every learner. A larger discount would recover most of that half percent but would lengthen the horizon over which the learners must propagate values, which Section VI-D identifies as their main difficulty.

D. Service level against travel

Re-solving with move costs between 0 and 8 traces the exact frontier between availability and travel (Fig. 4). At the chosen price of 0.1 the optimum sits on the flat part of the frontier. On layout 2, raising the price to 0.5 cuts travel from 120 to 107 moves per run while service falls from 0.969 to 0.968. If robot energy has a real cost to the store, the proposal's weight undervalues it; the frontier lets that decision be taken knowingly.

E. A bound from travel time

Each unit sold from shelf i must first be carried there. A reload, trip, restock and return carrying two units takes $2d_i + 2$ steps, where d_i is the shelf's distance from the depot, so each delivered unit costs at least $d_i + 1$ steps of robot time; splitting a load between shelves is never cheaper per unit. Expected sales from shelf i are at least $pT\,SL_i$ and cannot exceed its initial three units plus what is delivered. Hence

$$\sum_i (d_i + 1)\,\max\bigl(0,\ pT\,SL_i - 3\bigr) \le T + d_{\max} + 2, \qquad (8)$$

where the slack allows for the robot starting loaded and not returning after its last trip. Maximising the mean SL_i under this constraint is a small linear program. The bound binds only when both shelves are far from the depot (Table I). On layout 4, with shelves five and six cells away, no policy can exceed a service level of 0.807 and the optimum reaches 0.683. Elsewhere the bound is 0.98 or more, and what limits the optimum is the randomness of demand, not the robot's capacity.

V. LEARNING AGENTS AND PROTOCOL

A. Agents

Nine learners are compared. They are grouped by how they turn experience into an update, because that is the property the results turn on.

One-step temporal-difference control. Q-Learning [1] moves $Q(s,a)$ toward $r + \gamma \max_{a'} Q(s',a')$ and is off-policy. SARSA [2] uses $r + \gamma Q(s',a')$ for the action actually taken next and is on-policy.

Multi-step and Monte Carlo control. 4-step Sarsa replaces the one-step target with four observed rewards plus $\gamma^4 Q(s_{t+4}, a_{t+4})$ [3]. SARSA(λ) with $\lambda = 0.8$ applies every TD error to all recently visited pairs through accumulating eligibility traces, pruned once they fall below 10^{-3}. Every-visit Monte Carlo control moves Q toward the observed discounted return and never bootstraps inside an episode.

TABLE III
HYPERPARAMETERS. ALL LEARNERS: 120,000 STEPS, $\gamma = 0.95$, SAME MARKOV STATE INPUT.

Tabular (all five)	$\alpha = 0.15$ constant, $Q_0 = 0$, ε: $1 \to 0.05$
4-step Sarsa / SARSA(λ)	$n = 4$ / $\lambda = 0.8$, trace cut-off 10^{-3}
DQN, Double DQN	Adam $5 \cdot 10^{-4}$, batch 64, buffer 50k, target copied every 500 steps, Huber loss
REINFORCE + baseline	Adam $3 \cdot 10^{-3}$ (policy), 10^{-3} (value), one update per run, entropy 0.01
PPO	Adam $3 \cdot 10^{-4}$, rollout 2048, 10 epochs, batch 64, clip 0.2, GAE $\lambda = 0.95$, entropy 0.01
Networks	2×64 ReLU; rewards scaled by 0.1 in updates

Function approximation. DQN [7] learns Q_θ from a replay buffer against a periodically copied target network; Double DQN [9] lets the online network choose the next action and the target network evaluate it, to reduce overestimation. REINFORCE with a learned baseline [10] raises the log-probability of each action in proportion to its return minus a value estimate. PPO [12] reuses each batch for several epochs under a clipped probability-ratio objective, with advantages from generalised advantage estimation [11].

B. Shared settings

Every learner receives the same budget, 600 runs of 200 steps (120,000 environment steps), the same discount $\gamma = 0.95$, and the same five-number Markov state. Tabular agents use a 768×7 table; all networks have two hidden layers of 64 ReLU units and see the state scaled to $[0, 1]$. Value-based agents explore ε-greedily, with ε falling linearly from 1 to 0.05 over the first 450 runs and then held, so every method ends with the same consolidation phase. Table III lists the remaining settings. The DQN and PPO settings are the published defaults of each method [7], [12], scaled to this budget, and were not tuned, so that no algorithm benefits from a larger tuning effort than the others. REINFORCE is the one exception. It has no widely used default, and pooling ten runs per update left only 60 gradient steps in the budget, after which its policy was still at chance level. It therefore uses the textbook single update per run, with the policy learning rate chosen from $\{10^{-3}, 3 \cdot 10^{-3}, 10^{-2}\}$ on three layouts.

C. The time limit is not a terminal state

Because the store keeps running after step 200, treating the cut as terminal would teach every agent that the last few states of a run are worth nothing, which is false and, since the state does not include the clock, inconsistent with the same states seen earlier in a run. Following Pardo et al. [17], every target that reaches the limit bootstraps from the value of the state the run was cut in: the one-step and trace methods do so naturally, 4-step Sarsa bootstraps its last windows from $Q(s_T, a_T)$, Monte Carlo adds $\gamma^{T-t} Q(s_T, a_T)$ to its returns, REINFORCE adds $\gamma^{T-t} V(s_T)$, PPO's advantage estimate starts from $V(s_T)$, and DQN stores no done flag at all. Review II's DQN treated the cut as terminal, and its Monte Carlo and n-step agents dropped the tail; all three are corrected here.

D. Measurement and statistics

Each greedy policy is evaluated exactly (Section III-G) every 20 runs during training and once at the end. For the two policy-gradient methods both the greedy policy and the stochastic policy they actually learned are evaluated, since the two can behave very differently. Each experiment is replicated over the ten layouts, with one training run per layout; intervals are 95% t-intervals over layouts. Comparisons between training rewards are paired by layout, since the same store and the same random seed are used for each reward, and are tested with a paired t-test on the per-layout differences. This follows the advice of Henderson et al. [18] and Agarwal et al. [19] to report uncertainty across independent runs, not a single best run.

VI. RESULTS

Every learner was trained under every reward on every layout: nine learners, four rewards and ten layouts in the main grid (360 runs), plus the follow-up runs described below. Across all 730 runs, the simulated check agreed with exact evaluation as expected if both are correct (standardised difference: mean 0.24, standard deviation 1.01, $|z| > 2$ in 5.2% of runs).

A. Overall comparison

Table IV and Fig. 2 give the final normalised scores. Three groups stand out.

DQN and Double DQN are the strongest learners under every reward and come close to the optimum when trained on the Proposal reward (0.96 for both). They beat the best tabular method, SARSA(λ), on every reward (paired $p < 0.001$), and they learn fastest: half of the achievable improvement is reached after a median of 24,000–34,000 steps, against 72,000–76,000 for SARSA(λ) (Fig. 3). Double DQN is never significantly different from DQN ($p \geq 0.16$). Its correction for overestimation has little to correct in a problem this small.

PPO reaches 0.73–0.79, with much wider intervals than DQN: it solves some layouts almost perfectly and stalls on others. With the same 120,000 steps, an on-policy method that discards its data after each update is at a disadvantage against a replay-based one, and the learning curves are still rising when the budget ends.

The tabular learners spread from Q-Learning (0.17–0.27) to SARSA(λ) (0.63–0.73), in the order that the amount of credit assignment per update would suggest: multi-step and trace methods above one-step SARSA, and Q-Learning last. Monte Carlo control is the least sensitive to the reward and is never better than 0.43. REINFORCE with a baseline is the weakest learner by its greedy policy (0.04–0.22), but Section VI-E shows that this figure understates what it learned.

B. Operational metrics

Table V translates the scores into the six measures named in the project proposal, each defined so that it can be computed exactly: cumulative objective reward, service level (task

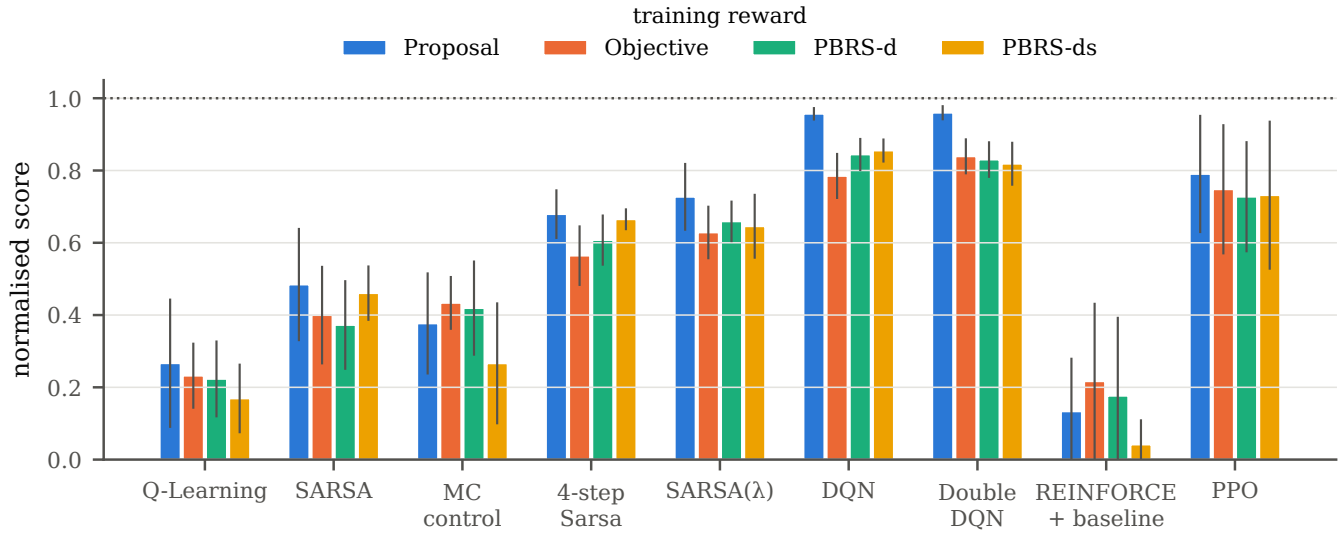


Fig. 2. Final normalised score of every learner under each training reward (mean and 95% CI over 10 layouts). The dotted line is the best achievable behaviour.

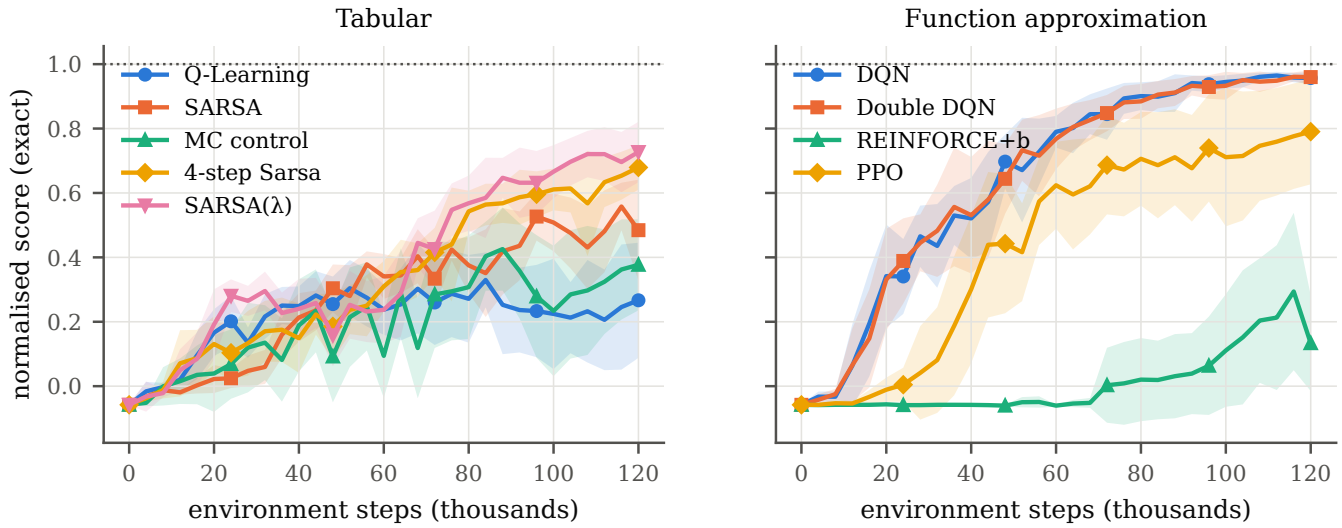


Fig. 3. Learning curves under the Proposal reward: the exact normalised score of the current greedy policy every 20 runs (mean and 95% CI over 10 layouts).

completion), travel (energy), mean stockout duration (response time, the number of empty shelf-steps per shelf that ran out), load factor (resource utilisation, units per restock as a share of capacity) and, in Fig. 3, convergence speed. DQN keeps shelves stocked 82% of the time against 84.5% for the optimum, and refills an empty shelf after 6.4 steps on average against 6.3. PPO shows how a mean can mislead: it has the largest share of proactive restocks, yet its mean stockout lasts 41 steps, because on the layouts where it stalls an empty shelf stays empty for most of the run. The tabular learners leave shelves empty for 11–67 steps.

C. Which reward trains best, and why

Section IV-B showed that the Proposal reward is slightly wrong at the optimum. Here it is the best training signal for seven of the nine learners, significantly so for 4-step Sarsa, SARSA(λ), DQN and Double DQN (Table IV). To find out which part of it helps, the restock bonus and the closer bonus were each added to the objective on their own (Table VI).

For DQN both terms help, by +0.12 and +0.10, and the reward containing both reaches 0.96, up from 0.78. Their potential-based counterparts (PBRS-ds and PBRS-d) recover roughly half of each gain while provably leaving the optimal policy unchanged. The price of the larger gain is visible in behaviour: DQN trained on the Proposal reward travels further

TABLE IV

FINAL NORMALISED SCORE BY LEARNER AND TRAINING REWARD (MEAN ± 95% CI OVER 10 LAYOUTS; 0 = RANDOM, 1 = BEST ACHIEVABLE). BOLD: BEST REWARD FOR THE LEARNER. *: DIFFERENT FROM OBJECTIVE, PAIRED t-TEST, $p < 0.05$.

learner	Proposal	Objective	PBRS-d	PBRS-ds
Q-Learning	**0.27 ± 0.18**	0.23 ± 0.09	0.22 ± 0.11	0.17 ± 0.10
SARSA	**0.48 ± 0.16**	0.40 ± 0.14	0.37 ± 0.12	0.46 ± 0.08
MC control	0.38 ± 0.14	**0.43 ± 0.07**	0.42 ± 0.13	0.27 ± 0.17
4-step Sarsa	**0.68 ± 0.07***	0.56 ± 0.08	0.61 ± 0.07	0.67 ± 0.03*
SARSA(λ)	**0.73 ± 0.09***	0.63 ± 0.07	0.66 ± 0.06	0.65 ± 0.09
DQN	**0.96 ± 0.02***	0.78 ± 0.06	0.84 ± 0.05	0.86 ± 0.03*
Double DQN	**0.96 ± 0.02***	0.84 ± 0.05	0.83 ± 0.05	0.82 ± 0.06
REINFORCE+b	0.13 ± 0.15	**0.22 ± 0.22**	0.18 ± 0.22	0.04 ± 0.07
PPO	**0.79 ± 0.16**	0.75 ± 0.18	0.73 ± 0.15	0.73 ± 0.21

TABLE V

EXPECTED METRICS PER 200-STEP RUN FOR LEARNERS TRAINED ON THE PROPOSAL REWARD, BESIDE THE OPTIMAL POLICY (MEANS OVER 10 LAYOUTS). J: OBJECTIVE; SL: SERVICE LEVEL; TRAVEL: MOVES; STOCKOUT: MEAN STOCKOUT DURATION (STEPS); PROACT.: SHARE OF RESTOCKS MADE BEFORE THE SHELF RAN OUT; LOAD: UNITS PER RESTOCK / CAPACITY; OPT.: PROBABILITY OF AN OPTIMAL ACTION ON STATES π^* VISITS.

	score	J	SL	travel	stockout	proact.	load	opt.
DP optimum π^*	0.99	-510	0.845	148	6.3	0.60	0.84	1.00
Q-Learning	0.27	-2076	0.353	56	66.9	0.58	0.81	0.60
SARSA	0.48	-1598	0.504	114	32.2	0.41	0.87	0.61
MC control	0.38	-1853	0.425	121	38.3	0.21	0.97	0.33
4-step Sarsa	0.68	-1187	0.633	127	14.3	0.33	0.90	0.53
SARSA(λ)	0.73	-1073	0.669	128	11.4	0.25	0.93	0.60
DQN	0.96	-589	0.821	154	6.4	0.50	0.87	0.89
Double DQN	0.96	-586	0.822	151	6.6	0.53	0.85	0.90
REINFORCE+b	0.13	-2387	0.257	104	126.6	0.70	0.62	0.68
PPO	0.79	-979	0.699	153	40.9	0.72	0.75	0.85

than the optimal policy on nine of the ten layouts. On layout 2, DQN and PPO both make about 140 moves per run where the optimum makes 120, and they sit to the right of the exact frontier (Fig. 4). The learners take the same loop that costs an optimal agent 0.3% in score, and in exchange they reach a far better policy within the budget. At this training budget the Proposal reward is therefore the better choice, and the exact audit says what it costs.

D. Why the tabular learners fall short

For the tabular learners, the reward explains less than it seems to. All five start from $Q_0 = 0$, and since the objective only ever charges, zero is above the optimal value of every state. Fig. 5 (left) measures how much of that optimism is left after training, as the gap between the learned values and V^* on the states the optimal policy visits. One-step Q-Learning and SARSA still sit about 37–39 above V^* after 120,000 steps: each update moves a value only a fraction α of the way toward a target that is itself too high. Eligibility traces and multi-step targets pass each correction back through many states at once and remove most of the gap (SARSA(λ): +7). Monte Carlo, which never bootstraps, ends below V^*. Across layouts, the remaining optimism goes with a lower score.

Wiewiora's result [14] shows why shaping cannot remove this gap. For a tabular learner, adding $\gamma\Phi(s') - \Phi(s)$ to every

TABLE VI

ONE PROPOSAL TERM AT A TIME (FINAL SCORE, MEAN OF 10 LAYOUTS). *: PAIRED $p < 0.05$ AGAINST OBJECTIVE.

learner	Obj.	+restock	+closer	Proposal	PBRS-d	PBRS-ds
Q-Learning	0.23	0.25	0.23	0.27	0.22	0.17
SARSA	0.40	0.42	0.40	0.48	0.37	0.46
4-step Sarsa	0.56	0.62	0.58	0.68*	0.61	0.67*
SARSA(λ)	0.63	0.71*	0.68*	0.73*	0.66	0.65
DQN	0.78	0.90*	0.88*	0.96*	0.84	0.86*

TABLE VII

FINAL SCORE ON THE OBJECTIVE REWARD FOR INITIAL VALUES Q_0 (MEAN OF 10 LAYOUTS; BOLD: BEST). FOR DQN, Q_0 IS SET THROUGH THE OUTPUT BIAS. LAST COLUMN: PROPOSAL REWARD WITH $Q_0 = 0$, FOR COMPARISON.

learner	0	−50	−100	−150	−200	−300	Prop.
Q-Learning	0.23	0.39	0.55	**0.69**	0.53	0.42	0.27
SARSA	0.40	0.47	0.64	**0.72**	0.57	0.38	0.48
4-step Sarsa	0.56	0.61	0.67	**0.75**	0.66	0.48	0.68
SARSA(λ)	0.63	0.70	0.75	**0.79**	0.70	0.52	0.73
MC control	**0.43**	0.37	0.37	0.35	0.41	0.40	0.38
DQN	0.78	–	–	**0.79**	–	–	0.96
Double DQN	**0.84**	–	–	0.78	–	–	0.96

reward is the same as starting from $Q_0(s, a) = \Phi(s)$. This was confirmed directly: with identical seeds, SARSA on PBRS-d and SARSA on the objective started from Φ visit the same states and keep tables that differ by exactly $\Phi(s)$ to 3×10^{-14} for all 600 runs. PBRS-d therefore only lowers the starting values by twice the distance to the target shelf, at most 12, against an optimism of about 50, which is why it barely moves the tabular scores. The Proposal reward helps them for a related reason: its bonuses raise the true values, so the zero start is only about 20 above them instead of 55.

The causal test is to keep the objective and change only Q_0 (Table VII, Fig. 6). Starting at −150 triples Q-Learning's score (0.23 to 0.69) and lifts every TD learner significantly above its $Q_0 = 0$ result ($p \leq 0.001$). It also beats the Proposal reward started at zero, significantly for Q-Learning, SARSA and 4-step Sarsa ($p < 0.01$) and narrowly for SARSA(λ) (0.79 against 0.73, $p = 0.10$). The effect has an inverted-U shape, with two failure modes that can be observed directly. Counting state–action visits during training on five layouts: with $Q_0 = 0$, the greedy action is the least-tried action in its state 9–16% of the time, so the policy is pulled toward the actions it knows least about. With $Q_0 = -300$ that never happens, but the greedy action has been tried thousands of times: the policy has locked onto early choices instead of comparing alternatives. The best constant, −150, lies close to the mean optimal value of the states an exploring agent visits (−137), between the largest (−19) and smallest (−197) optimal values. This is an observed coincidence, not a derived rule. Even at the best value, $Q_0 = -150$, the trained values sit 67–82 *below* V^*, so the greedy policy depends on how the leftover error is spread across the actions of a state more than on how accurate the values are. Monte Carlo is insensitive to Q_0, as expected for a method that never bootstraps.

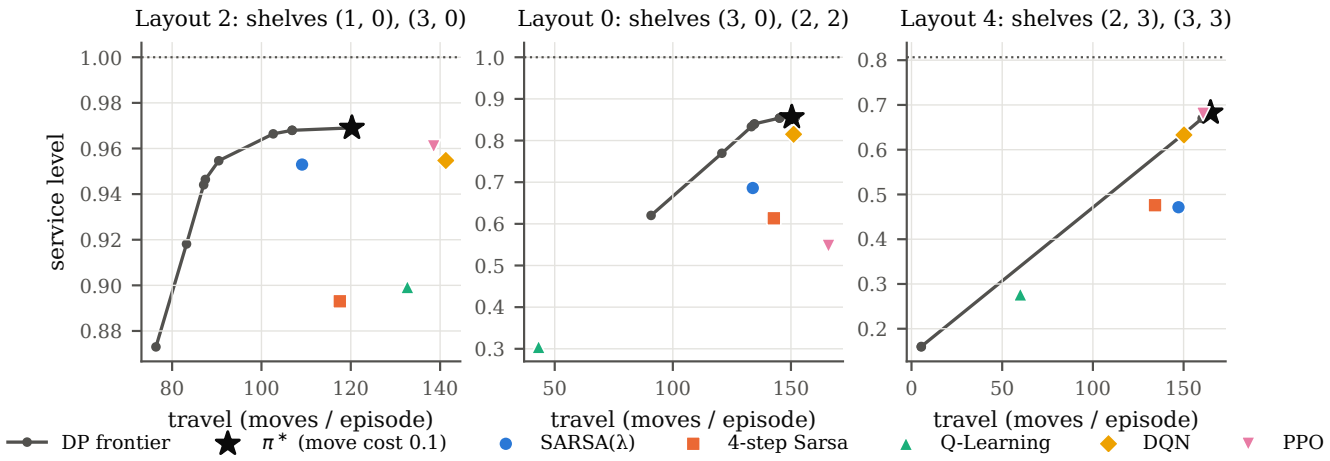


Fig. 4. Learned policies (Proposal reward) against the exact service–travel frontier of three layouts. The star is the optimum at the objective's travel price, and the dotted line is the travel-time bound. Points to the right of the frontier travel more than necessary for their service level.

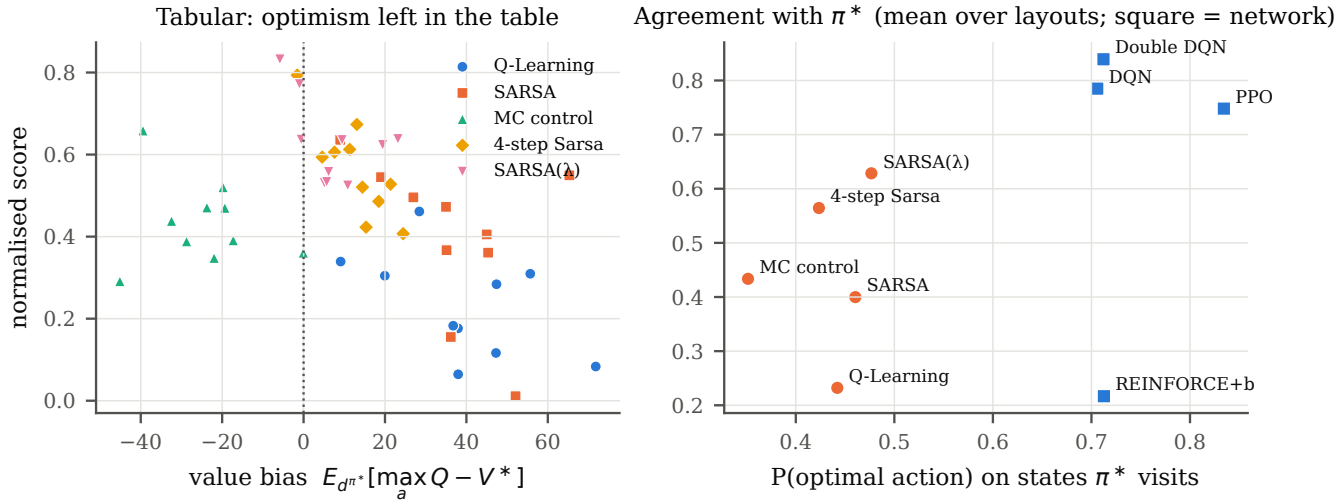


Fig. 5. Inside the learned policies (Objective reward). Left: optimism left in each Q-table after training (one point per layout). Right: probability of choosing an optimal action on the states the optimal policy visits, against the score (one point per learner).

None of this applies to DQN. Setting its output bias so that every initial value is −150 changes nothing for DQN (0.79 against 0.78, $p = 0.80$) and makes Double DQN slightly worse (0.78 against 0.84, $p = 0.02$). A network shares its parameters across states, so a few hundred updates rescale every output at once and no state keeps its initial value for long. For DQN, the gain from the Proposal reward is a genuine effect of its extra terms, as Table VI showed.

E. Greedy or stochastic policy-gradient agents

REINFORCE and PPO learn a stochastic policy, and both forms were evaluated exactly. For REINFORCE the difference is large: its stochastic policy scores 0.45–0.49, while the argmax of the same policy scores 0.27–0.41 lower under every reward ($p \le 0.012$). Reporting the argmax alone would understate what REINFORCE learned by more than half. For PPO the two forms agree on average (mean differences between −0.04 and +0.03), but on single layouts the gap reaches 0.62. Policy-gradient agents should therefore be evaluated with their stochastic policies.

VII. Discussion

A. Corrections to the earlier reviews

Measuring every policy against the optimum, on every layout, with identical budgets, changed several conclusions of Review II. They are listed here so the two reports can be read together.

The PPO "overfitting" case. Review II reported a PPO run that reached 81% completion during training but 18.6% on held-out demand, and attributed the gap to overfitting a demand sequence. That explanation cannot hold, since every training run already used a fresh demand seed. The training

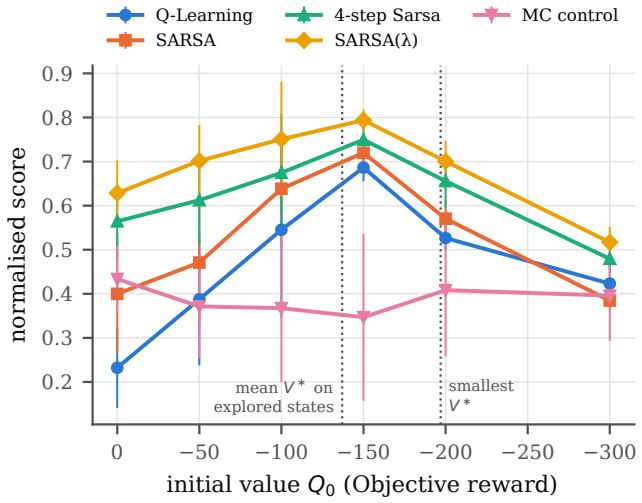


Fig. 6. Tabular learners on the Objective reward with different initial values (mean and 95% CI over 10 layouts). Dotted lines: mean optimal value of states an exploring agent visits, and the smallest optimal value.

curve was computed from the sampled stochastic policy, whereas the held-out figure came from its argmax, and the argmax of an undertrained stochastic policy can be very different from the policy itself. With both forms evaluated exactly, PPO's greedy and stochastic policies here differ by at most 0.04 in normalised score on average, although by up to 0.62 on a single layout, and REINFORCE's argmax scores 0.27–0.41 below its own stochastic policy (Section VI-E).

REINFORCE's energy result. REINFORCE was trained on 60-step runs while every other method used 200, so its travel and return figures were not comparable; its apparent energy efficiency reflected shorter runs and a policy that barely moved. All learners now use identical runs.

The 88.9% VI/PI agreement is fully explained by ties (Section IV); the two methods agree exactly.

The time limit. Review II's DQN treated step 200 as a terminal state, and its Monte Carlo and n-step agents ignored what lay beyond it; both bias the values of late states.

Layouts and seeds. Each training seed was also a different store, so seed-to-seed variance mixed algorithmic noise with layout difficulty, and the dynamic-programming reference existed for one layout only. Per-layout optima and the normalised score separate the two.

The tabular state carried a time-of-day bit that no part of the dynamics reads; it doubled the table without adding information.

B. Limitations

The store is small by design; that is what makes an exact optimum available. The tabular methods would not scale to a realistic store, and the absolute scores of the deep agents on this small problem say little about their behaviour on a large one. What should carry over is the method: build a version of the problem small enough to solve, and use it to check reward design and learner behaviour before scaling up.

Each layout received one training run per configuration, so the intervals combine layout and seed variation; with ten layouts this is enough to separate the large effects reported here, but not differences of a few hundredths. The deep agents used published default hyperparameters without tuning, which is fair across methods but probably leaves some performance unrealised for each. Finally, the best initial value was located with the help of the exact optimum; in a problem that cannot be solved, it would have to be estimated, for example from the returns of a simple hand-written policy, and that estimate was not tested here.

VIII. Conclusion

The small store was most useful as a measuring instrument. The exact optimum showed that the original reward's shaping term can be gamed and that its optimal policy does game it, but also that this costs very little, and it located the travel price at which availability starts to be traded for energy. Against the same optimum, DQN trained on the original reward came within a few percent of the best possible behaviour, and the extra terms that slightly bend the optimum are what made it learn that well within the budget. For the tabular learners the starting Q-value mattered more than the reward. Through Wiewiora's equivalence, the same fact explains why potential-based shaping barely changed them. Without exact values to compare the tables against, this would have looked like a difference between rewards.

In practice this suggests checking, on a solvable version of a problem, whether a shaped reward moves the optimum before trusting it; treating the initial value of a tabular learner as a design decision, since zero is optimistic whenever every reward is a cost; and evaluating policy-gradient agents with their stochastic policies.

The natural next steps are to make demand depend on the time of day, so that the clock in the observation carries information; to grow the store until exact solution becomes impossible, and test whether a model-free estimate of the initial value can replace the one read here from V^*; and to compare learned policies with the travel-time bound on larger layouts, where no exact optimum is available.

References

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[2] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Cambridge Univ. Eng. Dept., Tech. Rep. CUED/F-INFENG/TR 166, 1994.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[4] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton Univ. Press, 1957.

[5] R. A. Howard, *Dynamic Programming and Markov Processes*. Cambridge, MA, USA: MIT Press, 1960.

[6] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York, NY, USA: Wiley, 1994.

[7] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[8] H. van Hasselt, "Double Q-learning," in *Advances in Neural Information Processing Systems 23*, 2010, pp. 2613–2621.

[9] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artificial Intelligence*, 2016, pp. 2094–2100.

[10] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992.

[11] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[13] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th Int. Conf. Machine Learning (ICML)*, 1999, pp. 278–287.

[14] E. Wiewiora, "Potential-based shaping and Q-value initialization are equivalent," *J. Artificial Intelligence Research*, vol. 19, pp. 205–208, 2003.

[15] J. Randløv and P. Alstrøm, "Learning to drive a bicycle using reinforcement learning and shaping," in *Proc. 15th Int. Conf. Machine Learning (ICML)*, 1998, pp. 463–471.

[16] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," arXiv:1606.06565, 2016.

[17] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, PMLR vol. 80, 2018, pp. 4045–4054.

[18] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 3207–3214.

[19] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems 34*, 2021.

[20] J. Gijsbrechts, R. N. Boute, J. A. Van Mieghem, and D. J. Zhang, "Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems," *Manufacturing & Service Operations Management*, vol. 24, no. 3, pp. 1349–1368, 2022.

[21] R. N. Boute, J. Gijsbrechts, W. van Jaarsveld, and N. Vanvuchelen, "Deep reinforcement learning for inventory control: A roadmap," *European J. Operational Research*, vol. 298, no. 2, pp. 401–412, 2022.

[22] X. Liu, M. Hu, Y. Peng, and Y. Yang, "Multi-agent deep reinforcement learning for multi-echelon inventory management," *Production and Operations Management*, vol. 34, no. 7, pp. 1836–1856, 2025.

[23] S. Jullien, M. Ariannezhad, P. Groth, and M. de Rijke, "A simulation environment and reinforcement learning method for waste reduction," *Trans. Machine Learning Research*, 2023.

[24] Z. Zhu, S. Wang, and T. Wang, "Optimizing robotic mobile fulfillment systems for order picking based on deep reinforcement learning," *Sensors*, vol. 24, no. 14, art. 4713, 2024.

[25] K. Li *et al.*, "Research on reinforcement learning based warehouse robot navigation algorithm in complex warehouse layout," arXiv:2411.06128, 2024.